

# Query Dual Dynamics Delta: Read-Driven Modulation of Delta-Rule Writes in Fixed-Capacity Fast-Weight Memory

Dian Yu<sup>1\*</sup> and Haoyu Yang<sup>1</sup>

<sup>1\*</sup>University of New South Wales, Sydney, 2052, NSW, Australia.

\*Corresponding author(s). E-mail(s): [dian.yu2@student.unsw.edu.au](mailto:dian.yu2@student.unsw.edu.au);

Contributing authors: [z5575389@ad.unsw.edu.au](mailto:z5575389@ad.unsw.edu.au);

## Abstract

A fixed-capacity fast-weight memory is normally updated from the stream being written, although its usefulness is determined by a possibly different stream of future reads. This paper introduces Query Dual Dynamics Delta (**QD<sup>3</sup>**), an online operator that turns past queries into causal guidance for subsequent Delta writes. One state stores key-value associations; a second, low-rank state follows the dominant directions and energy of the query stream. Their only coupling is an interpretable scalar: the estimated historical query heat of the incoming write key. This heat increases the residual-correction rate, while a hard cap preserves the non-expansive single-address behavior of the normalized Delta rule. The construction adds  $\mathcal{O}(d_k r)$  state, is constant in sequence length, and recovers ordinary Delta exactly when query modulation is disabled. We evaluate the operator without end-to-end training so that memory behavior is measured directly. Hyperparameters are selected on eight validation seeds and frozen for 16 paired test seeds. **QD<sup>3</sup>** reduces normalized recall error by 31.0% on a concentrated spatial field, 18.7% on a source-token workload, 32.1% on a module-import graph, and 2.62% on held-out WikiText-2 token sequences. An estimator- and capacity-matched write-trace control establishes that the additional information comes from reads rather than extra state. Uniform-query, moving-hotspot, source, and rank controls further delimit the mechanism.

**Keywords:** fast-weight memory, Delta rule, linear attention, associative memory, workload adaptation, online subspace learning

# 1 Introduction

A compact memory is valuable precisely because it does not grow with the history it summarizes. The same constraint also creates competition: after sufficiently many key–value pairs have been written, improving one association can disturb another. Delta-style fast weights address part of this problem by writing only the residual that is still missing at the current key [1, 2]. They answer a local question—*what is wrong at this write address?*—but not the allocation question—*which addresses will matter when this memory is read?*

That distinction is immaterial when writes and reads have the same distribution. It becomes important for a long-running service, agent, map, or compressed sequence state in which the stored population is broad but the workload is concentrated. A small set of entities may be queried repeatedly even though every entity is written with comparable frequency. Write counts cannot reveal this demand. Recency cannot reveal it either. The read events themselves carry the missing signal.

We therefore treat a query as more than a transient probe. In Query Dual Dynamics Delta (QD<sup>3</sup>), reads learn a compact workload state, writes learn the key–value map, and a single non-negative heat value connects them. The core innovation is this causal cross-event allocation: accumulated read geometry controls the rate of *later* residual writes in a continuous, fixed-capacity memory. It neither performs an instantaneous query correction nor evicts discrete cached tokens. The two independently evolving states give the method its name.

The design follows four principles: only earlier reads may affect a write; cold-start behavior must remain ordinary Delta; the workload state must be much smaller than the memory it guides; and prioritization must remain inside the normalized Delta stability interval. These requirements lead directly to a causal low-rank query trace and a clipped effective write rate.

We study the operator separately from an end-to-end model so that its central hypothesis is directly testable: given identical writes, capacity, and evaluation samples, does read history improve later recall? The study uses a smooth two-dimensional field, two real-data-derived controlled workloads, and held-out WikiText-2 sequences [3]. Its decisive baseline matches the low-rank estimator, state size, update count, normalization, and tuning budget, but learns from write keys instead of query keys.

The contributions are:

- a causal read-driven Delta operator that uses accumulated query geometry to allocate corrective write rate;
- a low-rank query trace whose heat has a direct interpretation as discounted squared query–key alignment, with constant state in the stream length;
- a clipped update with exact reduction to ordinary Delta and a simple non-expansion guarantee at the written address; and
- a paired, validation-separated study that includes an estimator-matched source control, full-moment and weighted-linear references, rank and source ablations, uniform-query controls, and a changing workload.

**Table 1** Position of QD<sup>3</sup> among memory-management mechanisms. “Current” denotes information at the present sequence position; “history” denotes a separately accumulated statistic. Citations and fuller distinctions are given in Sect. 2.

| Method family             | Guidance                 | What changes                  | Difference from QD <sup>3</sup>        |
|---------------------------|--------------------------|-------------------------------|----------------------------------------|
| DeltaNet                  | current key and residual | one corrective write          | no read-history model                  |
| Gated DeltaNet            | current-token gates      | forgetting and/or writing     | learned current-token control          |
| DeltaProduct              | current-token residuals  | multiple Delta steps          | increases transition rank              |
| Q-Delta / QED             | current query            | correction or erase direction | instantaneous query-aware edit         |
| H <sub>2</sub> O / SnapKV | attention observations   | stored token positions        | evicts from a discrete KV cache        |
| QD <sup>3</sup> (ours)    | query history            | future residual-write rate    | cumulative continuous-state allocation |

## 2 Related work

### 2.1 Fast weights and Delta-based linear attention

Fast weights are parameters that change on the time scale of a sequence and store temporary associations [4, 5]. Kernelized linear attention can be written as a recurrent matrix update with state independent of sequence length [6]. The fast-weight-programming view makes the connection explicit: keys and values form programming instructions for a finite associative matrix [2]. Purely additive writing accumulates interference, whereas the Delta rule first subtracts the value already predicted at the current key and writes only the residual. Parallel DeltaNet later supplied a sequence-parallel implementation and demonstrated the rule at language-model scale [7].

Recent work has improved how the current token edits this matrix. Gated DeltaNet combines targeted Delta correction with a forgetting gate [8]; Gated DeltaNet-2 separates channel-wise erase and write gates [9]; and DeltaProduct applies several Delta steps per token, increasing the rank and expressivity of the transition [10]. These mechanisms decide how to edit from the current input and learned gates. QD<sup>3</sup> addresses a different variable: the empirical distribution of earlier reads.

The closest query-aware operators also use the current query more directly. Q-Delta mixes current key- and query-based prediction errors in one state transition [11]. Query-derived Erase Direction introduces a second, current-query erase direction to remove interference that a key-directed edit cannot reach [12]. Both are instantaneous query-write couplings at a sequence position. QD<sup>3</sup> instead accumulates a query second-moment surrogate over many read events and uses it to price future writes. The distinction is operational: a current query answers *what should be corrected now?*; a query history estimates *where future correction effort is likely to be useful?*

## 2.2 Access-aware retention and online subspaces

Read frequency has also been exploited in explicit key–value caches. H<sub>2</sub>O retains attention heavy hitters together with recent tokens [13], while SnapKV estimates important prompt positions from an observation window and compresses the discrete cache [14]. Those methods choose which token records remain available. A fast-weight matrix has no token list to evict after associations have been superposed, so access information must act through the continuous write rule instead. This is the gap addressed here.

The low-rank query state is updated by an Oja-type subspace rule [15]. Oja’s algorithm is a standard streaming route to principal directions, with established multi-component guarantees [16]. OjaKV recently used online Oja adaptation for low-rank compression of an explicit KV cache [17]. In QD<sup>3</sup>, no cached value is projected or discarded by this subspace. It approximates only the query workload and produces a scalar importance signal for the separate value memory.

At a statistical level, the motivation is related to covariate-shift correction: when training and test inputs differ, importance weighting aligns the fitted objective with the test distribution [18]. Exact importance weighting requires a density ratio. QD<sup>3</sup> neither estimates nor claims to recover that ratio. It uses a cheaper geometric proxy—discounted query energy near a key—that can be maintained online when only read addresses, not read-time target values, are available.

## 3 Problem formulation

Consider two interleaved event streams. A write event provides a key–value pair  $(\mathbf{k}_t, \mathbf{v}_t)$  with  $\mathbf{k}_t \in \mathbb{R}^{d_k}$  and  $\mathbf{v}_t \in \mathbb{R}^{d_v}$ . A query event provides an address  $\mathbf{q}_j \in \mathbb{R}^{d_k}$ . The indices  $t$  and  $j$  count writes and queries separately;  $j(t)$  denotes the number of queries observed before write  $t$ . All keys and queries are normalized to unit Euclidean norm in our experiments.

The fixed-capacity value state is  $\mathbf{M}_t \in \mathbb{R}^{d_v \times d_k}$ . A query is answered by

$$\hat{\mathbf{v}}(\mathbf{q}_j) = \mathbf{M}_t \mathbf{q}_j. \quad (1)$$

For a write, ordinary Delta first computes the current prediction and residual,

$$\hat{\mathbf{v}}_t = \mathbf{M}_{t-1} \mathbf{k}_t, \quad \mathbf{e}_t = \mathbf{v}_t - \hat{\mathbf{v}}_t, \quad (2)$$

then performs the rank-one correction

$$\mathbf{M}_t = \mathbf{M}_{t-1} + \eta \mathbf{e}_t \mathbf{k}_t^\top, \quad (3)$$

where  $\eta \geq 0$  is the write rate. Because  $\|\mathbf{k}_t\| = 1$ , Eq. (3) is a stochastic-gradient step on  $\frac{1}{2} \|\mathbf{v}_t - \mathbf{M} \mathbf{k}_t\|^2$ .

Let  $p_w$  denote the distribution of associations presented for writing and  $p_q$  the workload under which recall is evaluated. The desired linear state minimizes

$$\mathcal{L}_q(\mathbf{M}) = \frac{1}{2} \mathbb{E}_{(\mathbf{k}, \mathbf{v}) \sim p_q} [\|\mathbf{v} - \mathbf{M} \mathbf{k}\|^2]. \quad (4)$$

Plain Delta samples gradients under  $p_w$ . If  $p_q \neq p_w$ , even a well-tuned write process spends its updates according to the population presented, not the population later requested. An exact correction would multiply each gradient by  $p_q(\mathbf{k})/p_w(\mathbf{k})$ , but estimating a high-dimensional density ratio online is a substantially different problem. We seek a local signal that is available from queries alone and cheap enough to attach to every Delta memory.

## 4 Query Dual Dynamics Delta

### 4.1 The full query heat

Suppose first that storing a full  $d_k \times d_k$  query statistic is affordable. After query  $j$ , define the exponentially weighted, uncentered second moment

$$\mathbf{Q}_j = \lambda \mathbf{Q}_{j-1} + \mathbf{q}_j \mathbf{q}_j^\top, \quad 0 < \lambda \leq 1. \quad (5)$$

For any candidate write key  $\mathbf{k}$ , its unnormalized query heat is

$$\bar{h}_j(\mathbf{k}) = \mathbf{k}^\top \mathbf{Q}_j \mathbf{k} = \sum_{\ell=1}^j \lambda^{j-\ell} (\mathbf{q}_\ell^\top \mathbf{k})^2. \quad (6)$$

Equation (6) gives the central quantity a direct reading. Every earlier query votes for a new write in proportion to squared alignment in key space; the square makes the vote independent of sign, and  $\lambda$  determines how quickly old demand is forgotten. A key receives high heat when it lies in directions that have repeatedly been queried. No queried target value is required.

The full trace is useful as a reference, but its  $d_k^2$  state can rival or exceed the memory being guided. QD<sup>3</sup> therefore maintains a low-rank surrogate.

### 4.2 Low-rank query dynamics

Let  $\mathbf{U}_j = [\mathbf{u}_{j,1}, \dots, \mathbf{u}_{j,r}] \in \mathbb{R}^{d_k \times r}$  contain  $r \ll d_k$  adaptive directions and let  $\mathbf{a}_j \in \mathbb{R}_+^r$  store their discounted energies. When query  $\mathbf{q}_j$  arrives, we first project it into the current subspace,

$$\mathbf{z}_j = \mathbf{U}_{j-1}^\top \mathbf{q}_j. \quad (7)$$

The residual  $\mathbf{q}_j - \mathbf{U}_{j-1} \mathbf{z}_j$  is the part not reconstructed by that subspace. An Oja-type step rotates the directions toward this missing component:

$$\mathbf{U}_j = \mathbf{U}_{j-1} + \eta_u (\mathbf{q}_j - \mathbf{U}_{j-1} \mathbf{z}_j) \mathbf{z}_j^\top, \quad (8)$$

where  $\eta_u$  is the subspace rate. At the same event, projected energy is accumulated as

$$\mathbf{a}_j = \lambda \mathbf{a}_{j-1} + \mathbf{z}_j \odot \mathbf{z}_j. \quad (9)$$

Here  $\odot$  denotes elementwise multiplication. Equations (8) and (9) have complementary roles:  $\mathbf{U}_j$  tracks *which directions* queries occupy, while  $\mathbf{a}_j$  tracks *how much demand*

lies along each direction. The factorized moment surrogate is  $\mathbf{U}_j \text{diag}(\mathbf{a}_j) \mathbf{U}_j^\top$ , so the heat assigned to an incoming write key is

$$\bar{h}_j(\mathbf{k}_t) = \sum_{i=1}^r a_{j,i} (\mathbf{u}_{j,i}^\top \mathbf{k}_t)^2. \quad (10)$$

The magnitude of Eq. (10) depends on query count and geometry. To keep  $\beta$  interpretable across streams, we maintain one causal scale

$$s_j = \max \left\{ s_{j-1}, \sum_{i=1}^r a_{j,i} (\mathbf{u}_{j,i}^\top \mathbf{q}_j)^2, \epsilon \right\}, \quad h_j(\mathbf{k}) = \frac{\bar{h}_j(\mathbf{k})}{s_j}, \quad (11)$$

with a small numerical  $\epsilon > 0$ . This maximum-observed scale is a normalization device, not an additional learned parameter. It uses only information available at query  $j$ .

### 4.3 Read-driven rate and memory update

For write  $t$ , QD<sup>3</sup> uses the most recent query state  $j(t)$  and sets

$$\tilde{\eta}_t = \min \{ \eta [1 + \beta h_{j(t)}(\mathbf{k}_t)], c \}, \quad \beta \geq 0, \quad (12)$$

followed by

$$\mathbf{M}_t = \mathbf{M}_{t-1} + \tilde{\eta}_t (\mathbf{v}_t - \mathbf{M}_{t-1} \mathbf{k}_t) \mathbf{k}_t^\top. \quad (13)$$

The constant 1 in Eq. (12) matters. It retains the ordinary Delta update during cold start and for directions with no query evidence. The coefficient  $\beta$  controls only the additional effort assigned by the workload trace. Setting  $\beta = 0$  makes  $\tilde{\eta}_t = \eta$  whenever  $\eta \leq c$ , so Eq. (13) is exactly Eq. (3); this equality is also tested numerically in both implementations.

The cap  $c$  is a safeguard for the residual correction, not a cosmetic clamp. For unit-norm keys, the following property is immediate.

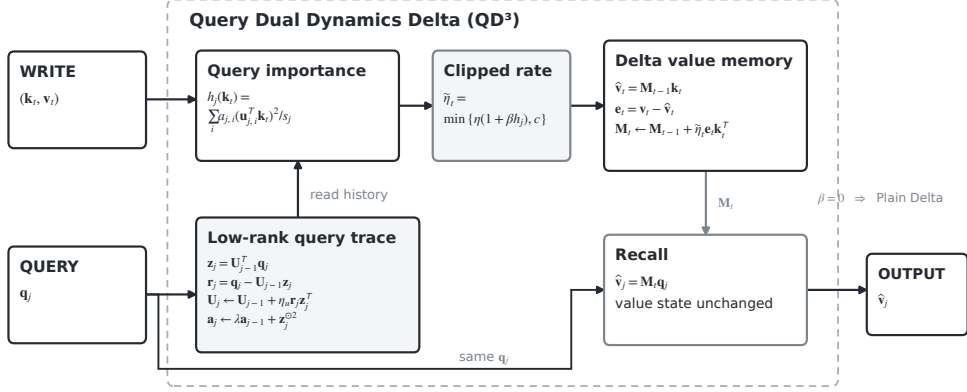
**Proposition 1** (Written-address non-expansion) *Let  $\|\mathbf{k}_t\| = 1$  and  $0 \leq \tilde{\eta}_t \leq 2$ . If  $\mathbf{e}_t$  is the pre-update residual in Eq. (2), the residual at the same key after Eq. (13) is*

$$\mathbf{e}_t^+ = (1 - \tilde{\eta}_t) \mathbf{e}_t, \quad (14)$$

and therefore  $\|\mathbf{e}_t^+\| \leq \|\mathbf{e}_t\|$ .

*Proof* Multiplying Eq. (13) by  $\mathbf{k}_t$  and using  $\mathbf{k}_t^\top \mathbf{k}_t = 1$  gives  $\mathbf{M}_t \mathbf{k}_t = \mathbf{M}_{t-1} \mathbf{k}_t + \tilde{\eta}_t \mathbf{e}_t$ . Subtracting from  $\mathbf{v}_t$  yields the stated identity. The norm inequality follows from  $|1 - \tilde{\eta}_t| \leq 1$  on  $[0, 2]$ .  $\square$

We use  $c = 1.9$ , leaving a margin below the endpoint. Proposition 1 is local to the written address; as with every shared fast-weight matrix, interference at other keys is still possible. It nevertheless explains why workload amplification is clipped before entering the Delta rule.



**Fig. 1** Query Dual Dynamics Delta. A query updates the low-rank workload state and is answered by the fixed-capacity value memory. A later write is scored against the accumulated query directions; the normalized heat modulates the ordinary residual Delta correction. The two states evolve on different event types and communicate only through the scalar heat. Setting  $\beta = 0$  exactly recovers Plain Delta.

Figure 1 summarizes the complete event flow. Query events update  $(\mathbf{U}, \mathbf{a}, s)$  and are answered from  $\mathbf{M}$  without directly changing it. Write events consult the frozen query state to update  $\mathbf{M}$  and do not train the query trace. This separation prevents write frequency from leaking into the quantity meant to represent read demand.

#### 4.4 State and computational cost

The value memory contains  $d_v d_k$  scalars. QD<sup>3</sup> adds  $d_k r + r + 1$  scalars for  $\mathbf{U}$ ,  $\mathbf{a}$ , and  $s$ , independent of the number of events. Its additional state fraction is therefore

$$\frac{d_k r + r + 1}{d_v d_k} \approx \frac{r}{d_v} \quad \text{when } d_k, d_v \text{ are not small.} \quad (15)$$

A query needs  $O(d_v d_k)$  work for recall and  $O(d_k r)$  for the trace update. A write retains the  $O(d_v d_k)$  Delta prediction and outer product and adds an  $O(d_k r)$  heat calculation. The full-trace reference adds  $d_k^2$  state and  $O(d_k^2)$  trace work. The low-rank construction is consequently most attractive when  $r \ll \min(d_k, d_v)$ .

## 5 Experimental design

### 5.1 Tasks and evaluation metric

The experiments isolate memory quality: there is no learned encoder, classifier, or end-to-end task head. This avoids attributing to the write rule gains that could instead come from additional model training.

### 5.1.1 Synthetic spatial field

Write locations are sampled uniformly from  $[0, 1]^2$ . A fixed 64-dimensional random Fourier map produces unit keys, and an eight-dimensional smooth harmonic field produces values with additive Gaussian noise of standard deviation 0.05. Query and evaluation locations are sampled independently. In the concentrated condition they come from two Gaussian hotspots; in the uniform condition they cover the square uniformly. A moving-hotspot condition switches to a disjoint pair halfway through the query stream. Each run has 3,000 writes and 5,000 evaluation samples. This controlled source is included to expose the mechanism under known independent distributions, rather than as an application benchmark.

### 5.1.2 Real-data-derived controlled workloads

Two structurally different streams are constructed from source-code text. The token workload contains 3,000 identifiers: short-window co-occurrence features are reduced to  $d_k = 128$  keys, long-window profiles give  $d_v = 64$  values, and observed token counts define read frequency. The module-import graph contains 578 modules: co-import structure gives  $d_k = 24$  keys and  $d_v = 48$  values, while import counts define the read distribution. Writes traverse every item in random order for three and six passes, respectively. These tasks retain real heavy-tailed workloads and geometry while allowing exact control of the read distribution. They are used for mechanism confirmation, not presented as end-to-end software applications.

### 5.1.3 WikiText-2 held-out token sequence

WikiText-2 is a public corpus of curated Wikipedia articles [3]. The training split fixes a 3,000-word vocabulary and builds a unit-normalized 256-dimensional short-window co-occurrence key for each word; normalized 64-dimensional long-window context profiles are the stored values. The write stream makes two independently shuffled passes over these 3,000 associations. Query events and evaluation items, however, come from a contiguous, disjoint window of the held-out validation or test token sequence: a query prefix is immediately followed by the scored portion. A shuffled-query control keeps the same query multiset and changes only its order. The prepared vocabulary covers 84.1% of held-out test tokens, and the most frequent 10% of vocabulary items account for 72.5% of covered test occurrences. Each run evaluates 3,000 held-out tokens.

For  $N$  evaluation associations  $(\mathbf{k}_n, \mathbf{v}_n)$ , lower normalized root mean squared error (NRMSE) is better:

$$\text{NRMSE} = \frac{\sqrt{N^{-1} \sum_{n=1}^N \|\mathbf{M}\mathbf{k}_n - \mathbf{v}_n\|^2}}{\sqrt{N^{-1} \sum_{n=1}^N \|\mathbf{v}_n\|^2}}. \quad (16)$$

Sampling evaluation items from the query workload makes Eq. (16) a Monte Carlo estimate of Eq. (4), normalized for comparability.

## 5.2 Baselines and controls

Plain Delta is Eq. (3). The main source control, *Write Trace*, is deliberately matched to QD<sup>3</sup>: it uses the same Oja update, rank choices, energy decay, heat normalization, number of trace events, rate equation, and validation budget, but supplies the contemporaneous write key to Eqs. (7)–(9). A difference between these two methods can therefore be attributed to the information source rather than additional state or computation.

The *Full Query Trace* uses Eq. (5) without low-rank approximation. The *Stationary Query Moment* receives the true stationary query second moment and uses the same scalar gain; it is an offline reference, not an upper bound. The *Weighted Linear* reference solves ridge-stabilized least squares under the evaluation weights. It is optimal only within a stationary linear-map class and is not a bound on arbitrary memory systems.

Mechanism ablations replace the learned low-rank trace with a coordinate-wise diagonal moment, the most recent query alone, or a fixed random rank-four subspace. Rank is swept over  $r \in \{1, 2, 4, 8, 16\}$ . Uniform queries test the case in which the query source carries no distributional preference beyond broad coverage. Query-frequency power  $\alpha \in \{0, 0.25, 0.5, 0.75, 1, 1.25\}$  continuously changes the controlled discrete workloads from uniform ( $\alpha = 0$ ) to and beyond their observed skew ( $\alpha = 1$ ).

## 5.3 Selection, pairing, and implementation checks

Pilot seeds inspected during development are excluded. For the synthetic and controlled workloads, eight new validation seeds (2001–2008) select hyperparameters and 16 locked test seeds (10001–10016) are evaluated once. WikiText-2 uses validation seeds 4001–4008 and disjoint test seeds 30001–30016. Every method within a seed receives the same write order, query sequence, and evaluation samples. Each test seed occupies a disjoint block of the held-out token sequence before its contiguous window is selected.

The main search uses  $\eta \in \{0.02, 0.05, 0.1, 0.2, 0.4\}$ ,  $\beta \in \{1, 3, 10, 30, 100\}$ , and  $r \in \{2, 4, 8\}$  for low-rank methods. WikiText-2 uses  $\eta \in \{0.02, 0.05, 0.1\}$ ,  $\beta \in \{1, 3, 10, 30\}$ , and  $r \in \{1, 2, 4\}$ . Each method is tuned separately on validation NRMSE. Fixed operator settings are  $\eta_u = 0.05$ ,  $\lambda = 0.999$ ,  $c = 1.9$ , and one query event per three writes.

We report test means, sample standard deviations, and paired differences with two-sided 95% Student-*t* confidence intervals over 16 seeds. The pre-specified primary comparisons are QD<sup>3</sup> versus Plain Delta and versus Write Trace; full-trace comparisons are secondary. Complete validation grids, selected settings, per-seed observations, manifests, and file hashes accompany the manuscript. Unit tests cover finite outputs, batched/individual agreement, deterministic replay, exact  $\beta = 0$  reduction, source isolation, and agreement among NumPy, Torch, and CUDA implementations to within  $2 \times 10^{-4}$ .

**Table 2** Blind-test NRMSE on non-uniform workloads, reported as mean (sample standard deviation) over 16 paired seeds. Gain is the relative reduction of QD<sup>3</sup> from Plain Delta. Paired QD<sup>3</sup>–Write Trace confidence intervals are reported in the surrounding text.

| Workload              | Plain         | Write trace   | QD <sup>3</sup>      | Full trace    | Gain (%) |
|-----------------------|---------------|---------------|----------------------|---------------|----------|
| Spatial, concentrated | .7305 (.0450) | .7347 (.0534) | <b>.5040</b> (.0455) | .5047 (.0467) | 31.01    |
| Source-token profiles | .7718 (.0391) | .6421 (.0243) | <b>.6278</b> (.0164) | .6143 (.0096) | 18.66    |
| Module-import graph   | .8923 (.0219) | .6447 (.0282) | <b>.6056</b> (.0190) | .6035 (.0171) | 32.13    |
| WikiText-2, ordered   | .1719 (.0157) | .1698 (.0160) | <b>.1674</b> (.0147) | .1673 (.0148) | 2.62     |

## 6 Results

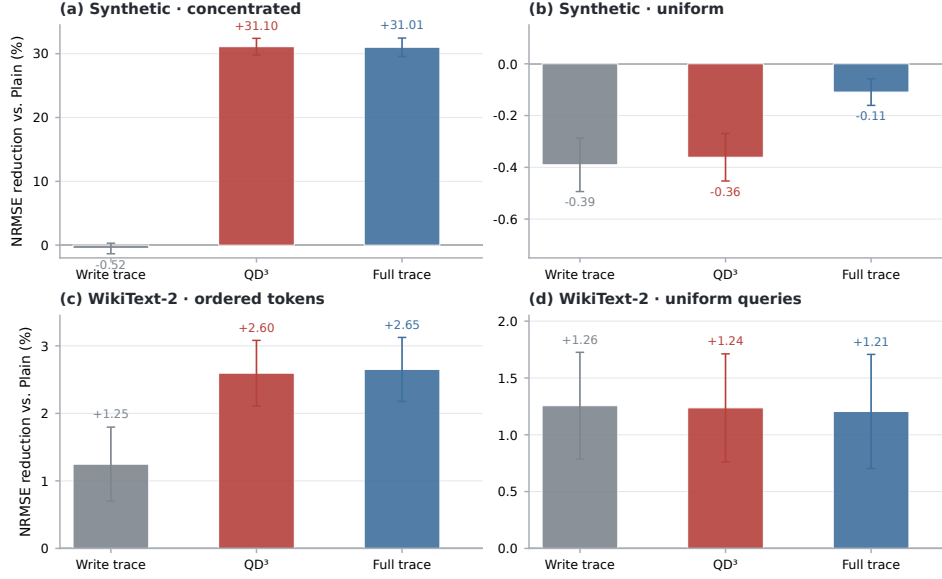
### 6.1 Concentrated and held-out workloads

Table 2 reports the four principal non-uniform workloads. On the controlled spatial field, QD<sup>3</sup> lowers NRMSE from 0.7305 to 0.5040, a 31.0% reduction. Write Trace does not improve Plain Delta, whereas the low-rank query trace is statistically indistinguishable from the full online query moment (paired difference  $-0.0007$ , 95% CI  $[-0.0042, 0.0029]$ ). Thus, the spatial gain is neither a general consequence of adaptive rates nor dependent on storing a dense moment.

The two real-data-derived workloads reproduce the effect across text statistics and graph structure. QD<sup>3</sup> reduces error by 18.7% on token profiles and 32.1% on the module-import graph. Write frequency is informative here: Write Trace already obtains reductions of 16.8% and 27.8%. The remaining paired improvements of QD<sup>3</sup> are nevertheless consistent: the QD<sup>3</sup>–Write Trace differences are  $-0.0144$  (95% CI  $[-0.0216, -0.0071]$ ; 14/16 wins) and  $-0.0391$  (95% CI  $[-0.0460, -0.0321]$ ; 16/16 wins), respectively. Reads therefore supply information beyond a strong frequency-based baseline.

On held-out WikiText-2 sequences, the effect is smaller but remains paired and repeatable. QD<sup>3</sup> reduces NRMSE by 2.62% relative to Plain Delta and by 1.42% relative to Write Trace; the paired difference is  $-0.0024$  (95% CI  $[-0.0034, -0.0014]$ ; 14/16 wins). Its mean differs from the full trace by only  $8.6 \times 10^{-5}$ , with a confidence interval spanning zero. Shuffling the order of the same query multiset changes QD<sup>3</sup>’s gain from 2.62% to 2.46%; frequency and key geometry, rather than a fragile local token ordering effect, account for most of the result.

Figure 2 places the central positive conditions beside their uniform controls. Under uniform spatial queries, QD<sup>3</sup> and Write Trace are indistinguishable (paired difference  $-0.00021$ , 95% CI  $[-0.00084, 0.00042]$ ), and both are slightly worse than Plain Delta. In the uniform WikiText-2 workload, both adaptive low-rank methods improve Plain Delta by about 1.2%, but their difference is only  $3.1 \times 10^{-5}$  NRMSE. The same source equality appears on uniform token and import workloads: the paired QD<sup>3</sup>–Write Trace confidence intervals both include zero. The relevant boundary is therefore precise: when queries carry no extra allocation information, the query trace loses its advantage over its matched write-derived counterpart. Our claim concerns workload-aware allocation, not universal improvement under identical event distributions.

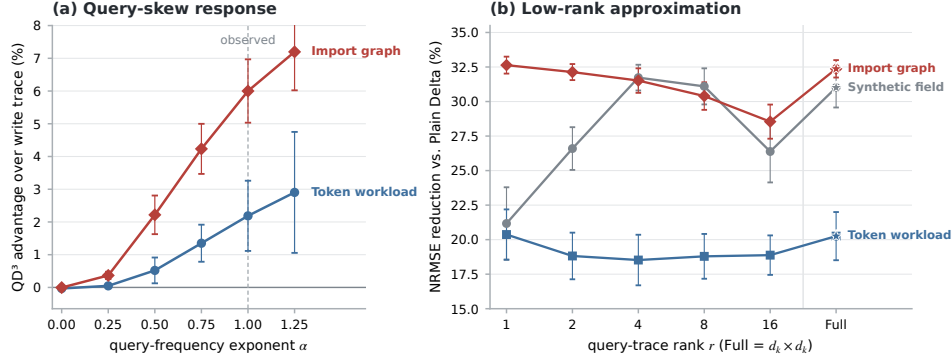


**Fig. 2** Blind-test NRMSE reduction relative to Plain Delta (mean and paired 95% confidence interval; 16 seeds; each panel has its own labeled vertical scale). On the controlled spatial field, query-derived traces improve concentrated-query recall while the matched write trace does not; their distinction disappears under uniform queries. On WikiText-2, contiguous held-out queries give QD<sup>3</sup> a 2.62% reduction, whereas QD<sup>3</sup> and Write Trace are nearly identical under uniform queries. Lower NRMSE is better.

## 6.2 Where the gain comes from

The query-skew sweep in Fig. 3a tests the proposed cause continuously rather than only at two endpoints. At  $\alpha = 0$ , QD<sup>3</sup> has no measurable advantage over Write Trace. As read demand becomes more concentrated, the advantage grows monotonically to 2.90% on token profiles and 7.20% on the import graph at  $\alpha = 1.25$ . This is the pattern expected if the second state measures workload mismatch: changing the query distribution changes the value of query history while leaving the write population fixed.

Figure 3b and Table 3 examine representation. A fixed random rank-four subspace is ineffective, so state count alone is not sufficient. A diagonal trace works when demand is aligned with input coordinates but fails on the rotated spatial features. The most recent query captures some locality, yet accumulated geometry is consistently stronger on the spatial task. Learned ranks between one and four recover essentially all of the full-trace benefit on the two real-data-derived workloads; the spatial field needs four directions for its best test mean. Larger rank is not automatically better because validation chooses one global  $(\eta, \beta)$  and finite-stream subspace estimates become noisier. The result supports compression of the workload statistic, not a claim that one rank is universal.



**Fig. 3** Mechanism tests on blind seeds. (a) The additional NRMSE reduction of QD<sup>3</sup> over the estimator- and capacity-matched Write Trace grows with the query-frequency exponent and begins near zero for uniform queries. (b) A small adaptive query subspace recovers the full online second-moment result; optimal rank is task dependent and selected only on validation seeds.

**Table 3** Ablation results as percentage NRMSE reduction from Plain Delta. “Random” uses a fixed random rank-four subspace; “best low rank” reports the best tested rank by blind-test mean for interpretation, while all headline ranks in Table 2 were selected on validation data.

| Workload              | Diagonal | Last query | Random | Best low rank     | Full trace |
|-----------------------|----------|------------|--------|-------------------|------------|
| Spatial, concentrated | −0.82    | 16.09      | −0.64  | 31.73 ( $r = 4$ ) | 30.91      |
| Source-token profiles | 18.30    | 20.12      | 1.70   | 20.53 ( $r = 1$ ) | 20.40      |
| Module-import graph   | 27.50    | 30.71      | 3.00   | 32.65 ( $r = 1$ ) | 32.37      |

### 6.3 Changing demand

The moving-hotspot experiment changes the query center halfway through the stream and evaluates the final region. Plain Delta obtains  $0.7528 \pm 0.0241$  NRMSE and Write Trace  $0.7630 \pm 0.0346$ . QD<sup>3</sup> reaches  $0.5600 \pm 0.0390$ , a 25.6% reduction from Plain Delta. It also outperforms the dense full trace ( $0.5814 \pm 0.0484$ ) by 0.0214 NRMSE, with paired 95% CI  $[-0.0329, -0.0100]$ . The low-rank bottleneck and exponential energy decay together favor dominant recent directions, which is useful when demand moves. We interpret this result as evidence of online tracking, not as a general guarantee that low rank will always outperform a full moment.

## 7 Discussion

### 7.1 Design interpretation

The results identify more than an adaptive-rate effect because Write Trace supplies that matched comparison. They support the intended information flow: reads estimate demand, the estimate allocates correction to later writes, and the value state remains responsible for association. Equation (4) specifies the population the memory should serve, while Eq. (6) supplies an online geometric proxy using addresses alone.

This proxy is not a density-ratio estimator or a replacement for the weighted-linear solution; its contribution is to make read-aware allocation causal, inspectable, and maintainable in  $O(d_k r)$  state. Uniform controls complete the interpretation: matched read and write traces agree when their information is interchangeable and separate when access concentrates elsewhere in key space.

## 7.2 Application value and deployment scope

The appropriate deployment setting has three observable properties: capacity remains fixed while associations continue to arrive, reads occur before some later writes, and access is more concentrated than the stored population. The closest use case is a linear-attention or fast-weight sequence layer, whose recurrent matrix is already a continuous superposition with no token list to evict. Other natural cases are persistent agent or retrieval memory, where repeated requests reveal semantic demand, and robot or sensor maps, where revisited regions should receive more accurate later updates. In all three cases the access summary remains  $O(d_k r)$  rather than growing into a log.

These conditions can be checked before deployment from capacity pressure and access statistics. A build-once store has no causal read signal, while uniform access offers no source-specific allocation advantage. The next step is therefore not to broaden the claim, but to integrate the operator into chunkwise training kernels and measure downstream language, retrieval, and embodied-memory behavior.

## 8 Conclusion

Query Dual Dynamics Delta turns past access into a compact causal signal for future storage. A low-rank trace learns query directions and energy, a normalized and clipped heat modulates the ordinary residual Delta write, and  $\beta = 0$  exactly recovers the base rule. Across controlled and held-out workloads, the query source improves recall when demand is concentrated and consistently exceeds an estimator-matched write source. The central design lesson is consequently precise: in a continuously updated fixed-capacity memory, reading can guide later write allocation without retaining an ever-growing access history.

## Statements and Declarations

**Competing interests.** The authors have no relevant financial or non-financial interests to disclose.

**Funding.** The authors received no financial support for the research, authorship, or publication of this article.

**Author contributions.** Dian Yu conceived the method, designed and conducted the experiments, analyzed the results, and drafted the manuscript. Haoyu Yang reviewed and revised the manuscript. Both authors read and approved the final manuscript.

**Ethics approval.** Not applicable. The study uses public text and locally derived software-structure statistics and does not involve interaction with human participants.

**Consent to participate.** Not applicable.

**Consent for publication.** Not applicable.

**Data availability.** WikiText-2 is publicly available with the corpus introduced by Merity et al. [3]. The exact prepared arrays used for the reported experiments are included in the reproducibility package accompanying this submission.

**Code availability.** Source code for all memory operators, validation and test evaluation, statistical analysis, and implementation checks is included with this submission and will be deposited in a public repository on acceptance.

## References

- [1] Widrow, B., Marcian E. Hoff, J.: Adaptive switching circuits. In: IRE WESCON Convention Record, vol. 4, pp. 96–104 (1960). <https://isl.stanford.edu/~widrow/papers/c1960adaptiveswitching.pdf>
- [2] Schlag, I., Irie, K., Schmidhuber, J.: Linear transformers are secretly fast weight programmers. In: Proceedings of the 38th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 139, pp. 9355–9366 (2021). <https://proceedings.mlr.press/v139/schlag21a.html>
- [3] Merity, S., Xiong, C., Bradbury, J., Socher, R.: Pointer sentinel mixture models. arXiv preprint arXiv:1609.07843 (2016)
- [4] Schmidhuber, J.: Learning to control fast-weight memories: An alternative to dynamic recurrent networks. Neural Computation 4(1), 131–139 (1992) <https://doi.org/10.1162/neco.1992.4.1.131>
- [5] Ba, J., Hinton, G., Mnih, V., Leibo, J.Z., Ionescu, C.: Using fast weights to attend to the recent past. arXiv preprint arXiv:1610.06258 (2016)
- [6] Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F.: Transformers are RNNs: Fast autoregressive transformers with linear attention. In: Proceedings of the 37th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 119, pp. 5156–5165 (2020). <https://proceedings.mlr.press/v119/katharopoulos20a.html>
- [7] Yang, S., Wang, B., Zhang, Y., Shen, Y., Kim, Y.: Parallelizing linear transformers with the delta rule over sequence length. In: Advances in Neural Information Processing Systems, vol. 37 (2024). <https://arxiv.org/abs/2406.06484>
- [8] Yang, S., Kautz, J., Hatamizadeh, A.: Gated delta networks: Improving Mamba2 with delta rule. In: International Conference on Learning Representations (2025). <https://arxiv.org/abs/2412.06464>
- [9] Hatamizadeh, A., Choi, Y., Kautz, J.: Gated DeltaNet-2: Decoupling erase and write in linear attention. arXiv preprint arXiv:2605.22791 (2026)
- [10] Siems, J., Carstensen, T., Zela, A., Hutter, F., Pontil, M., Grazi, R.: DeltaProduct: Increasing the expressivity of DeltaNet through products of householders.

arXiv preprint arXiv:2502.10297 (2025)

- [11] Park, S., Kim, S., Park, N.: Q-Delta: Beyond key-value associative state evolution. arXiv preprint arXiv:2606.08804 (2026)
- [12] Gupta, D., Das, A., Gupta, D.: The query knows what to forget: A second erase direction for linear attention. arXiv preprint arXiv:2608.13668 (2026)
- [13] Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., Chen, B.:  $H_2O$ : Heavy-hitter oracle for efficient generative inference of large language models. arXiv preprint arXiv:2306.14048 (2023)
- [14] Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., Chen, D.: SnapKV: LLM knows what you are looking for before generation. In: Advances in Neural Information Processing Systems, vol. 37 (2024). <https://doi.org/10.52202/079017-0722>
- [15] Oja, E.: Simplified neuron model as a principal component analyzer. Journal of Mathematical Biology **15**, 267–273 (1982) <https://doi.org/10.1007/BF00275687>
- [16] Huang, D., Niles-Weed, J., Ward, R.: Streaming  $k$ -PCA: Efficient guarantees for Oja’s algorithm, beyond rank-one updates. In: Proceedings of Thirty Fourth Conference on Learning Theory. Proceedings of Machine Learning Research, vol. 134, pp. 2463–2498 (2021). <https://proceedings.mlr.press/v134/huang21a.html>
- [17] Zhu, Y., Yang, D.H., Amiri, M.M., Murugesan, K., Pedapati, T., Chen, P.-Y.: OjaKV: Context-aware online low-rank KV cache compression with Oja’s rule. arXiv preprint arXiv:2509.21623 (2025)
- [18] Sugiyama, M., Nakajima, S., Kashima, H., Buenau, P.V., Kawanabe, M.: Direct importance estimation with model selection and its application to covariate shift adaptation. In: Advances in Neural Information Processing Systems, vol. 20 (2007)